SOCIAL EAGLE
learn. build. operate

WEEK 1 · DAY 4

Build a Streamlit Student Grade Manager

Assignment | learn. build. operate

1. Problem Statement

Take the grade manager you wrote on Day 2 out of the terminal and into the browser with **Streamlit**. Same grading scale, same calculations, a form to add a student, a table of results, and the class figures on screen. One thing will surprise you: Streamlit **reruns your whole script** every time anyone touches anything, and a plain Python list does not survive that.

Tools: Python, Streamlit

2. Requirements

- Keep all your code in a single file named `grade_app.py`, run with `streamlit run`.
- Use **Streamlit only**, no Flask, Django, FastAPI or Gradio.
- A form to add a student: name and mark.
- Keep the student list in `st.session_state` so it survives the reruns.
- Show all students in a **table** with name, mark and grade.
- Show the **class average, highest and lowest** as metrics on the page.
- Reject marks outside 0to100 in the interface, and use the **same grading scale as Day 2**.

> **Think it through**
>
> Try it the obvious way first: a plain `students = []` at the top. Add one student, then add another, and watch what happens to the first. Understanding **why** is the whole point of today: it is the one thing that separates a Streamlit app from the script you wrote on Day 2.

3. Steps to complete

These are hints, not the full answer. Fill in the blanks yourself.

Step 1: Set up

```
mkdir grade-app && cd grade-app
python -m venv venv
# macOS / Linux                    # Windows (PowerShell)
source venv/bin/activate           venv\Scripts\Activate.ps1
pip install streamlit
```

Step 2: Build the page

Bring over `get_grade()` from Day 2 unchanged: the logic does not need rewriting, only the way you store and show things does.

```
import streamlit as st

if "students" not in st.session_state:
    st.session_state.students = []

with st.form("add"):
    name = st.text_input("Name")
    mark = st.number_input("Mark", min_value=0, max_value=100)
    if st.form_submit_button("Add"):
        ...

if st.session_state.students:
    st.table(...)
    st.metric(...)
```

Step 3: Run it

```
streamlit run grade_app.py     # not python grade_app.py
```

Add four or five students and check the table and the metrics update each time.

4. Final output

```
Student Grade Manager

  Name [ Divya ]   Mark [ 83 ]   (Add)

  Name     Mark   Grade          Average  Highest  Lowest
  Priya      92     A              77.7      92      59
  Arun       78     C
  Divya      83     B
```

5. What to submit

- Your `grade_app.py` on **GitHub**, with the link shared in **WhatsApp**.
- A screenshot of the app in the browser with several students added.
- One line on what went wrong before you used `session_state`, posted in the **community**.